Deep generative learning

(learning to generate data)

Samples from a data distribution —train→ neural network

neural network —sample→ generated data

↑ mimic the training data distribution

The landscape of deep generative learning

1. Variational Autoencoders (VAE)
2. energy based models.
3. Autoregressive models
4. normalizing flows.
5. Denoising diffusion models.

Denoising Diffusion probabilistic models

forward process, (in each step) variance (eq. 1)

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I)$$

- x_t: current step
- x_{t-1}: previous step
- normal distribution
- $\sqrt{1-\beta_t}\, x_{t-1}$: mean ($\mu$)
- β_t: variance

β (variance): a small & badly rigid (e.g., 0.001) positive

So, $\mu: \rightarrow \sqrt{1-\beta} \simeq 0.999$

forward process →

$$x_0 \rightarrow \cdots x_{t-1} \longrightarrow x_t \rightarrow \cdots \rightarrow x_T$$

from the above equation (eq. 1) we can define a joint distribution from all samples starts from x_1 to x_T

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}) \quad \text{(eq. 2)}$$

→ the joint distribution of all samples in forward process.

The above process (forward process) called markov chain process.

Diffusion Kernel: (forward process)

we can jump from step 1 to step t or T by defining:

$\bar{\alpha}_t$ → scaler of forward t

$$\bar{\alpha}_t = \prod_{s=1}^{t} (1-\beta_s) \qquad \text{(eq:3)}$$

So ↓

$$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}\, x_0, (1-\bar{\alpha}_t) I) \qquad \text{(eq:4)}$$

Gaussian (Diffusion Kernel)

mean(μ) variance

- The diffusion kernel diffuse the input image or data from time 0 to time t.

- using reparameterization trick we can draw samples from x_t

for sampling: $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{(1-\bar{\alpha}_t)}\, \epsilon \qquad \text{(eq:5)}$

where $\epsilon \sim \mathcal{N}(0, I)$

- β_t values schedule (i.e., the noise schedule) is designed such that

$$\bar{\alpha}_T \to 0 \quad \text{and} \quad q(x_T \mid x_0) \approx \mathcal{N}(x_T; 0, I) \qquad \text{(eq. 5)}$$

$\bar{\alpha}_T$ will converge to zero

standard normal distribution.

the above equation means that at the end of the forward process the data ~~loss~~ will have a normal distribution.

- what happens to a distribution in the forward diffusion process?

So far, we discussed the diffusion kernel $q(x_t \mid x_0)$ but what about $q(x_t)$?

$$q(x_t) = \int q(x_0, x_t)\, dx_0 = \int q(x_0)\, q(x_t \mid x_0)\, dx_0 \qquad \text{(eq. 7)}$$

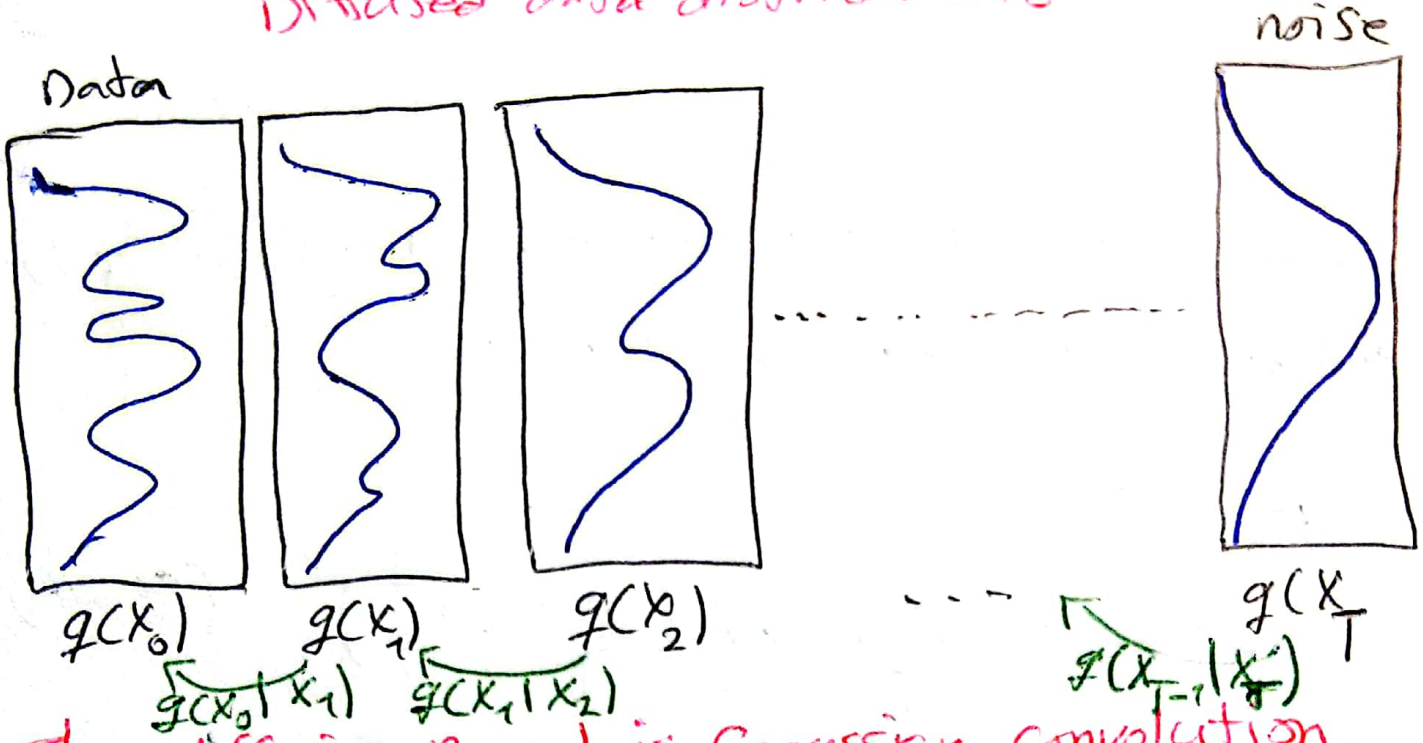
$q(x_t)$: diffused data dist.

$q(x_0, x_t)$: joint dist.

$q(x_0)$: input data dist

$q(x_t \mid x_0)$: ~~kernel~~ diffusion kernel

Diffused data distributions



The diffusion Kernel is Gaussian convolution.

- we should have in mind that we do not have access to input data distribution and intermediate diffuse data distribution, in practice we only have samples from data distribution (we do not have access to the explicit form of the property density function of data distribution), so in order to generate data from diffused data distribution we can just simply sample from training data x_0 and then we can just follow the forward diffusion process and sample x_T and x_0 in order to draw samples from x_T and this is basically the principle of ancestral sampling.

reverse process

~~Sampl~~ starts with sampling from noise ($g(X_T)$) and follows the backward process.

generation:

sample $X_T \sim N(X_T; 0, I)$ (eq: 8)

Iteratively sample $X_{t-1} \sim g(X_{t-1} | X_t)$

true denoising dist.

- The problem here is that in general denoising distribution X_t is intractable (we do not have access to it), here we can use bayes rule to show that this distribution is ~~proportional~~ proportional to the product of the marginal data distribution of X_{t-1} times diffusion kernel at step t.

(eq-9)

$$g(X_{t-1} | X_t) \propto \underbrace{g(X_{t-1})}_{\text{intractable} \uparrow} g(X_t | X_{t-1})$$

So the product above is intractable

- can we approximate $q(x_{t-1} | x_t)$?

yes, we can use a normal distribution if β_t is small in each forward diffusion step.

this means we can train a model which can approximate $q(x_{t-1} | x_t)$ which is actually a normal distribution if in the forward process we use very small noises.

this means the reverse process also can be approximate using a normal distribution.

- reverse denoising process (generative):

parametric reverse denoising process

$$p(x_T) = \mathcal{N}(x_T; 0, I)$$ → standard normal distribution (eq. 10)

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I)$$ (eq. 11)

distribution of the noise (last step of forward process)

trainable network (u-net, denoising autoencoder)

join distribution of the reverse denoising process.

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} | x_t) \quad \text{(eq. 12)}$$

Learning Denoising Model (Variational upper bound)

for training, we can form variational upper bound that is commonly used for training variational autoencoders:

$$E_{q(x_0)}\left[-\log p_\theta(x_0)\right] \leq E_{q(x_0) q(x_{1:T}|x_0)}\left[\cdots\right] \quad \text{(eq. 13)}$$

intractable

expectation over training data distribution

expectation over samples drawn from the forward process

$$\hookrightarrow -\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \Big] =: L$$

no denominator: joint distribution of the samples of the full trajectory which is given by the denoising model.

denominator: likelihood of the samples generated by the encoder

based on Sohl-Dickstein et al. and Ho et al. the variational upper bound can be decomposed to several terms:

(eq. 14)

$$L = E_q\Big[\underbrace{D_{KL}(q(x_T|x_0)\,\|\,p(x_T))}_{L_T} + \sum_{t>1} D_{KL}(q(x_{t-1}|x_t, x_0)\,\|\,p_\theta(x_{t-1}|x_t)) - \underbrace{\log p_\theta(x_0|x_1)}_{L_0}\Big]$$

L_{t-1}

L_T: distribution of the last step on forward process which can be ignored due to its nature (it is defined as the distribution of the noise which is a normal distribution).

L_0: reconstruction term. which measures the likelihood of input clean image given the noise image at the first step of the reverse process

L_{t-1} : $q(x_{t-1} | x_t, x_0)$ is the tractable posterior distribution

$$q(x_{t-1} | x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I),$$

where $\tilde{\mu}_t(x_t, x_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t} x_0 + \frac{\sqrt{1-\beta_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} x_t$ and $\tilde{\beta}_t := \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$

since both $q(x_{t-1} | x_t, x_0)$ and $p_\theta(x_{t-1} | x_t)$ are normal distributions, the KL (kernel divergence) has a simple form:

(eq. 14)

$$L_{t-1} = D_{KL}\left(q(x_{t-1} | x_t, x_0) \,\|\, p_\theta(x_{t-1} | x_t)\right) =$$

$$= E_q\left[\frac{1}{2\sigma_t^2} \| \tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t) \|^2\right] + C$$

difference between the mean of the ~~denoising~~ trajectory and the other denoising

C constant

recall that $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{(1-\bar{\alpha}_t)}\, \epsilon$. Ho et al. observe that as (eq. 95)

$$\tilde{\mu}_t(x_t, x_0) = \frac{1}{\sqrt{1-\beta_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon \right)$$

they propose to represent the mean of the denoising model using a noise-prediction network:

(eq. 10)

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{1-\beta_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right)$$

with this parameterization ~~use~~

(eq. 10)

$$L_{t-1} = E_{x_0 \sim q(x_0),\, \epsilon \sim N(0,I)} \left[\frac{\beta_t^2}{2\sigma_t^2 (1-\beta_t)(1-\bar{\alpha}_t)} \right] \| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, t) \|^2 + C$$

$\Leftrightarrow$ A_t ; x_t

~~the term β_t~~

the time dependent λ_t ensures that the training objective is weighted properly for the maximum data likelihood training. However, this weight is often very large for small t's.

Ho et al. (2020) observed that simply setting $\lambda_t = 1$ improves sample quality. So, they propose to use:

$$L_{simple} = E_{x_0 \sim q(x_0), \epsilon \sim N(0,I), t \sim U(1,T)} \left[\| \epsilon - \epsilon_\theta \left(\underbrace{\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1+\bar{\alpha}_t}\, \epsilon}_{x_t}, t \right) \|^2 \right]$$

Summary

Training and Sample Generation

Algorithm 1 (training)

1. repeat
2. $x_0 \sim q(x_0)$ draw a batch of samples from training data distribution.
3. $t \sim \text{uniform}(\{1, \ldots, T\})$ uniformly sample from time steps from 1 to T
4. $\epsilon \sim N(0, I)$ draw some random noise with the same dimensions as the input data.

⑤ Take gradient descent step on

$$\nabla_\theta \| \epsilon - \epsilon_\theta(\underbrace{\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon}_{\text{Parametrization trick to generate sample at time step } t}, t) \|^2$$

and we give this sample to noise prediction network and we train the noise prediction network to predict the noise ~~of~~ which was used to generate the noisy image at time step t

⑥ until converged.

→ generate data using reverse process.

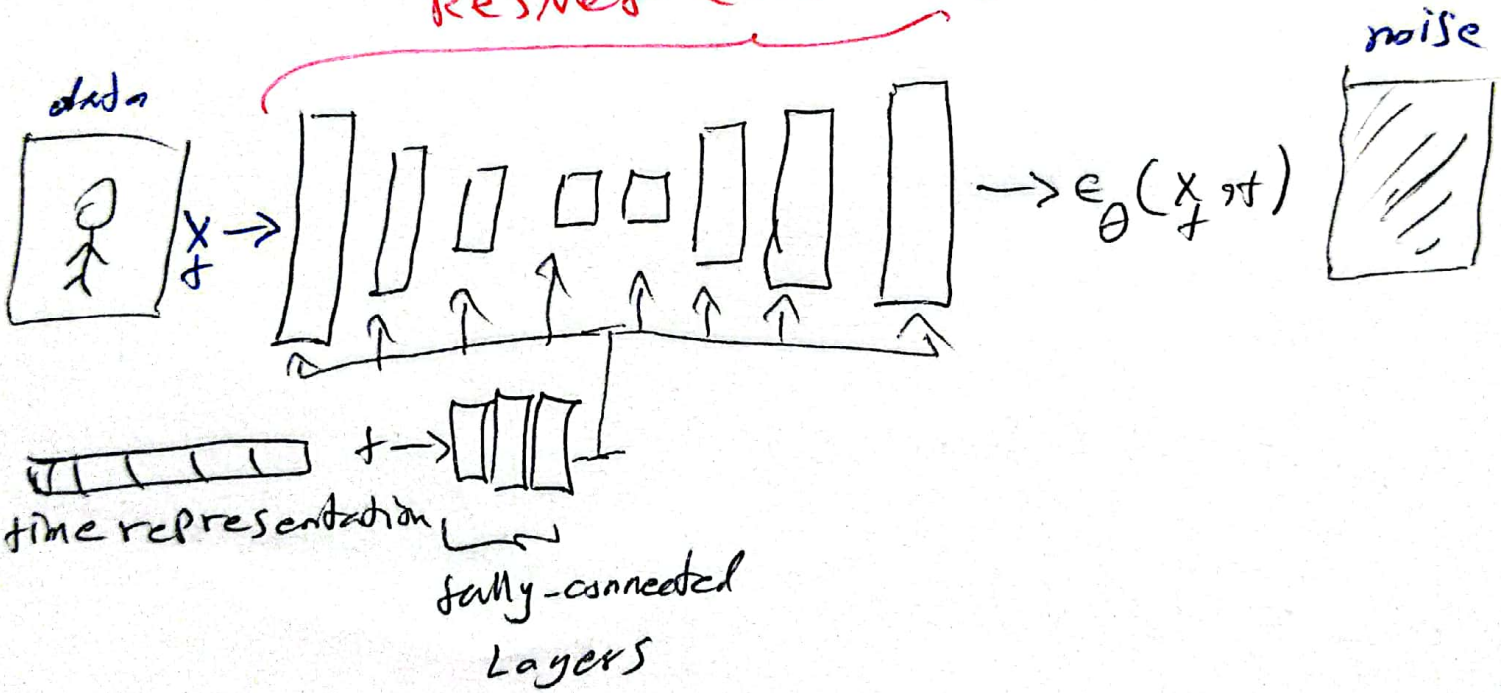
Algorithm 2 (Sampling)

1. $x_T \sim \mathcal{N}(0, I)$ sample from noise (standard normal distribution)
2. for $t = T, \ldots, 1$ do
3. $z \sim \mathcal{N}(0, I)$ Sample noise from standard distribution in each step
4. $x_{t-1} = \underbrace{\frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right)}_{\text{mean of the noisy model}} + \sigma_t z$
5. end for
6. return x_0 generated data.

Implementation ~~Architect~~ Considerations

network Architectures

- diffusion models often use U-Net architectures with ResNet blocks and self-attention layers to represent $\epsilon_\theta(x_t, t)$



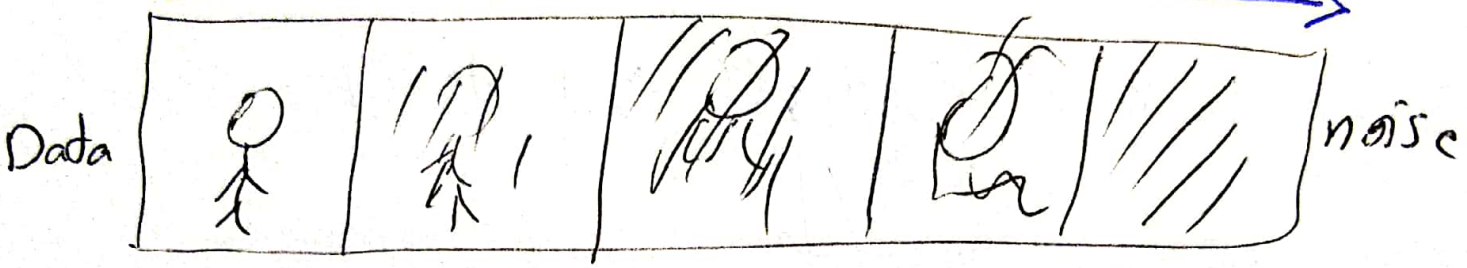
time representation: sinusoidal positional embeddings or random Fourier features.

time features are fed to the residual blocks using either simple spatial addition or using adaptive group normalization layers.

Diffusion Parameters

"noise schedule"

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I)$$



$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I)$$

Above, β_t and σ_t^2 control the variance of the forward diffusion and reverse denoising processes respectively

- often a linear schedule is used for β_t, and σ_t^2 is set equated to β_t.

what happens to an image in the forward diffusion process?

- recall that sampling from $q(x_t|x_0)$ is done using

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{(1-\bar{\alpha}_t)}\, \epsilon \quad \text{where } \epsilon \sim \mathcal{N}(0, I)$$

Parametrization trick

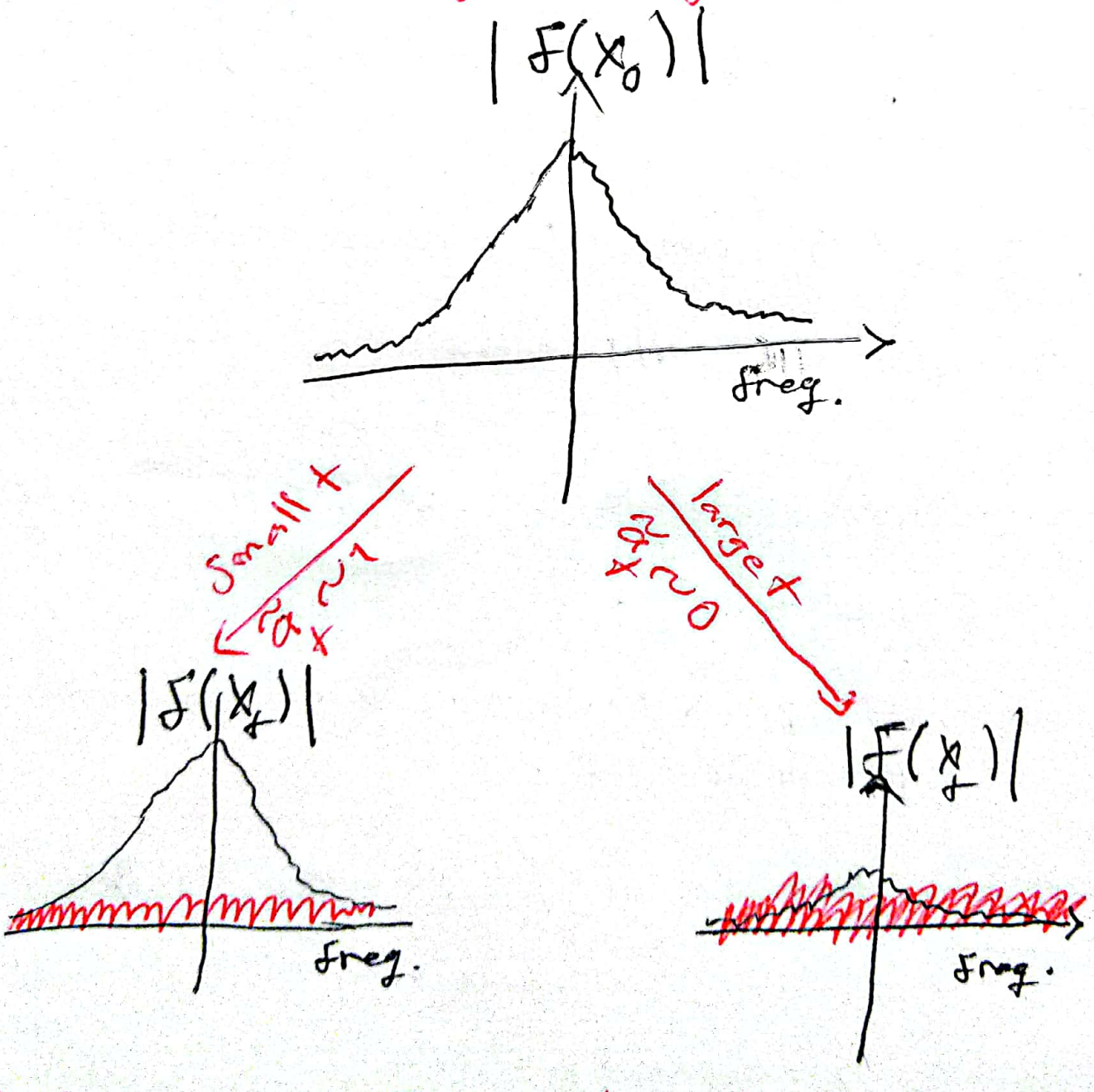
$$X_t = \sqrt{\bar{\alpha}_t}\, X_0 + \sqrt{(1-\bar{\alpha}_t)}\, \epsilon$$ diffused sample

↓ fourier transform

frequency domain

$$F(X_t) = \sqrt{\bar{\alpha}_t}\, F(X_0) + \sqrt{(1-\bar{\alpha}_t)}\, F(\epsilon)$$

represent the image in frequency domain



- in the forward diffusion, the high frequency content is perturbed faster.

- in reverse process in the first steps model tries to generate low-frequency content (course content) (overall structure) and in the last steps the model generates high-frequency content (low-level details)

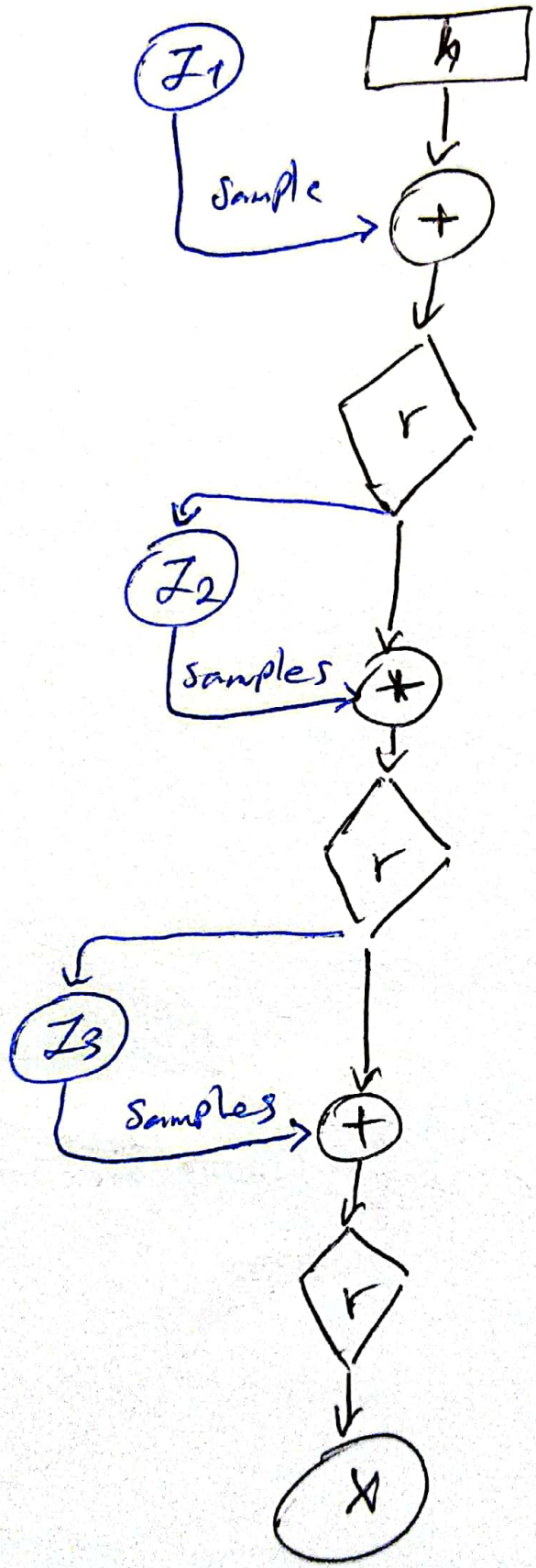
"Connection to VAEs"

- diffusion models can be considered as a special form of hierarchial VAEs.

However in diffusion models:

- The encoder (forward process) is fixed.
- The latent variables have the same dimension as the data.
- The denoising model is shared across different time steps.
- the model is trained with some reweighting of the variational bound.

"hirarchical VAEs"



Summary

denoising diffusion probabilistic models

- The model is trained by sampling from the forward diffusion process and training a denoising model to predict the noise.

score-based generative modeling with ~~diffusion~~ differential equations.

forward diffusion process (fixed) ⟶

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I)$$ forward process

↓ sampling

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \underbrace{\sqrt{\beta_t}\, \mathcal{N}(0, I)}_{\text{noise}}$$ parametrization trick

↓

$$= \sqrt{1-\beta(t)\Delta t}\, x_{t-1} + \sqrt{\beta(t)\Delta t}\, \mathcal{N}(0, I) \qquad (\beta_t := \beta(t)\Delta t)$$ time step

↓ if Δt is small enough

$$\approx x_{t-1} - \underbrace{\frac{\beta(t)\Delta t}{2} x_{t-1}}_{\text{correction term}} + \underbrace{\sqrt{\beta(t)\Delta t}\, \mathcal{N}(0, I)}_{\text{noise}}$$ ~~taylor~~ (taylor expression)

$$dX_t = -\frac{1}{2}\beta(t) X_t \, dt + \sqrt{\beta(t)}\, dw_t$$

stochastic differential equation (SDE) describing the diffusion in infinitesimal limit.

ordinary differential equation (ODE):

$$\frac{dX}{dt} = f(X,t) \quad \text{or} \quad dX = f(X,t)\,dt$$

Analytical solution: $X(t) = X(0) + \int_0^t f(X,\tau)\, d\tau$

iterative numerical solution: $X(t+\Delta t) \approx X(t) + f(X(t),t)\Delta t$

stochastic differential equation (SDE):

$$\frac{dX}{dt} = \underbrace{f(X,t)}_{\text{drift coefficient}} + \underbrace{g(X,t)\, w_t}_{\text{diffusion coefficient}}$$

← wiener process (Gaussian white noise)

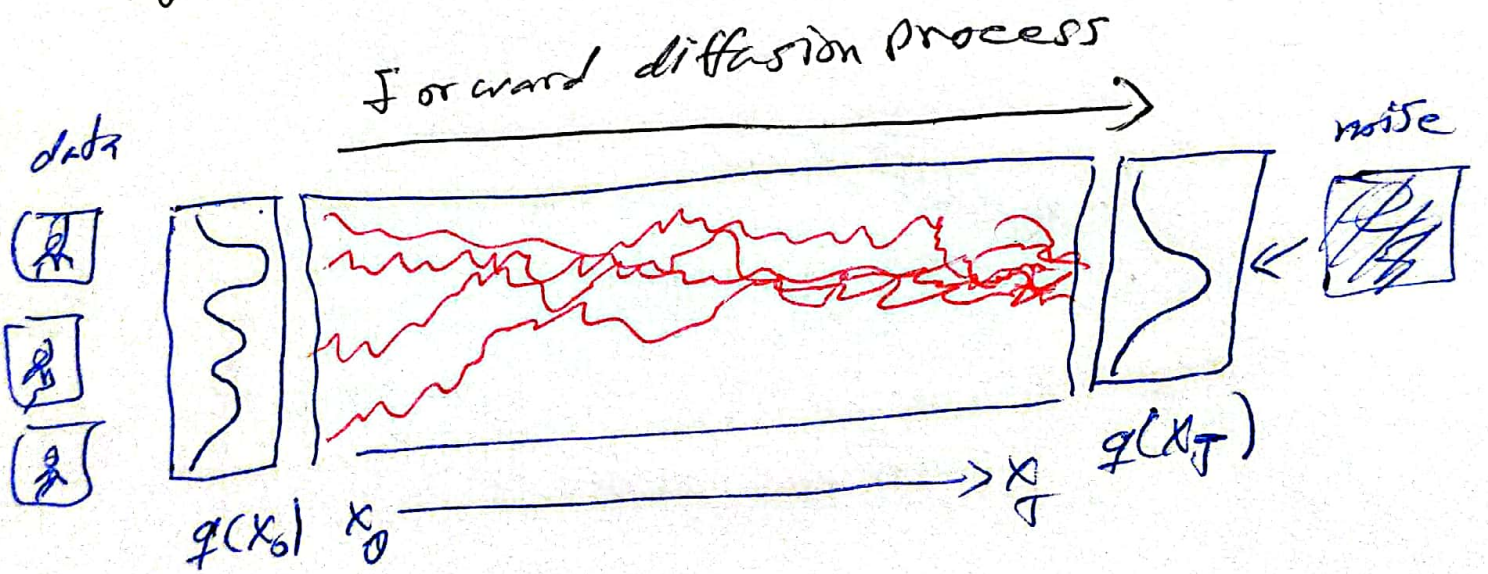
$$\left(dX = f(X,t)\,dt + g(X,t)\, dw_t\right) \quad \text{at} \quad \boxed{}$$

numerical solution

$$X(t+\Delta t) \approx X(t) + f(X(t), t)\Delta t + \sigma(X(t), t)\sqrt{\Delta t}\,N(0, I)$$

noise

Forward diffusion process as stochastic differential equation.



forward diffusion SDE:

$$dx_t = -\frac{1}{2}\beta(t)\,x_t\,dt + \sqrt{\beta(t)}\,dw_t$$

drift term (pulls towards mode) — diffusion term (injects noise)

- Special case of more general SDEs used in generative diffusion models:

$$dX_t = f(t) X_t \, dt + g(t) \, dw_t$$

reverse generative diffusion SDEs

drift term

score function

$$dX_t = \left[-\frac{1}{2} \beta(t) X_t - \beta(t) \nabla_{X_t} \log q_t(X_t) \right] dt + \sqrt{\beta(t)} \, d\bar{w}_t$$

diffusion term

Score function: gradient of the logarithm of the marginal ~~diffused density~~ probability density of the data

- Simulate reverse diffusion process: Data generation from random noise!

~~Hour~~

How to get the score function?

$$\Delta_{x_t} \log q_t(x_t)$$

- naive idea, learn model for the score function by direct regression

diffusion time t, diffusion data x_t, neural network

$$\min_\theta \mathbb{E}_{t\sim u(0,T)} \mathbb{E}_{x_t\sim q_t(x_t)} \| s_\theta(x_t, t) - \nabla_{x_t} \log q_t(x_t) \|_2^2$$

score of diffusion data (marginal)

- bad $\nabla_{x_t} \log q_t(x_t)$ (score of the marginal diffusion density $q_t(x_t)$) is not tractable!

Denoising score matching

- Instead, diffuse individual data points x_0. diffused $q_t(x_t|x_0)$ is tractable!

$$q_t(x_t|x_0) = \mathcal{N}(x_t; \gamma_t x_0, \sigma_t^2 I)$$

$$\gamma_t = e^{-\frac{1}{2}\int_0^t \beta(s)ds}$$

$$\sigma_t^2 = 1 - e^{-\int_0^t \beta(s)ds}$$

neural network

denoising score matching:

$$\min_\theta \underbrace{E_{t\sim u(0,T)}}_{\text{diffusion time } t} \underbrace{E_{x_0\sim q_0(x_0)}}_{\text{data sample } x_0} \underbrace{E_{x_t\sim q(x_t|x_0)}}_{\text{diffused data sample } x_t} \| s_\theta(x_t,t) \hookrightarrow - \nabla_{x_t} \log q_t(x_t|x_0) \|_2^2$$

score of diffused data sample

- after expectations: $s_\theta(x_t,t) \approx \nabla_{x_t} \log q_t(x_t)$!

- reparametrized sampling:

$$x_t = \gamma_t x_0 + \sigma_t \epsilon \qquad \epsilon \sim N(0,I)$$

- score function: $\nabla_{x_t} \log q_t(x_t|x_0) = -\nabla_{x_t} \frac{(x_t-\gamma_t x_0)^2}{2\sigma_t^2}$

$$= -\frac{x_t-\gamma_t x_0}{\sigma_t^2}$$

$$= \frac{\gamma_t x_0+\sigma_t\epsilon-\gamma_t x_0}{\sigma_t^2}$$

$$= \frac{-\epsilon}{\sigma_t}$$

-neural network models:

$$s_\theta(x_t, t) := -\frac{\epsilon_\theta(x_t, t)}{\sigma_t}$$

$$\min_\theta E_{t\sim U(0,T)} E_{x_0\sim q_0(x_0)} E_{\epsilon\sim N(0,I)} \frac{1}{\sigma_t^2} \| \epsilon - \epsilon_\theta(x_t, t) \|_2^2$$

↳ Denoising score matching

Variance reduction and numerical stability

$$\min_\theta E_{t\sim U(0,T)} E_{x_0\sim q_0(x_0)} E_{\epsilon\sim N(0,I)} \frac{\lambda(t)}{\sigma_t^2} \| \epsilon - \epsilon_\theta(x_t, t) \|_2^2$$

- notice $\sigma_t^2 \to 0$, as $t \to 0$. loss heavily amplified when sampling t close to 0 (for $\lambda(t) = \beta(t)$).

high variance!

1. Train with small time cut-off η ($\approx 10^{-5}$):

$$\min_\theta E_{t\sim\mathcal{U}(\eta,T)} E_{x_0\sim g_0(x_0)} E_{\epsilon\sim\mathcal{N}(0,I)} \frac{f(t)}{\sigma_t^2} \| \epsilon - \epsilon_\theta(x_t, t) \|_2^2$$

2. Variance reduction by Importance Sampling:

Importance sampling distribution: $r(t) \propto \frac{f(t)}{\sigma_t^2}$

$$\min_\theta E_{t\sim r(t)} E_{x_0\sim g_0(x_0)} E_{\epsilon\sim\mathcal{N}(0,I)} \frac{1}{r(t)} \frac{f(t)}{\sigma_t^2} \| \epsilon - \epsilon_\theta(x_t, t) \|_2^2$$

"Probability Flow ODE"

consider reverse generative diffusion SDE:

$$dx_t = -\frac{1}{2}\beta(t)\left[x_t + 2\nabla_{x_t} \log g_t(x_t)\right]dt + \sqrt{\beta(t)}\, d\bar{w}_t$$

in distribution equivalent to "probability flow ODE:

$$dx_t = -\frac{1}{2}\beta(t)\left[x_t + \nabla_{x_t} \log q_t(x_t)\right] dt$$

(solving this equation (ODE) results in the same $q_t(x_t)$ when initializing $q_T(x_T) \approx \mathcal{N}(x_T; 0, I)$)

- Probability flow ODE as neural ODE or continuous normalizing flow (CNF):

$$dx_t = -\frac{1}{2}\beta(t)\left[x_t + s_\theta(x_t, t)\right] dt$$

$$\left(\frac{dx_t}{dt} = -\frac{1}{2}\beta(t)\left[x_t + s_\theta(x_t, t)\right]\right)$$

"advantages"

The method above enable us to use advanced ODE solvers.

2. Deterministic encoding and generation

(Semantic image ~~interpolation~~ interpolation, etc.)

↳ continuous changes in latent space (x_T) result in continuous, semantically meaningful changes in data space (x_0)!

3. log-likelihood computation (instantaneous change of variables):

$$\log p_0(x_0) = \log p_T(x_T) - \int_0^T \mathrm{Tr}\left(\frac{1}{2}\beta(t)\frac{\partial}{\partial x_t}\left[x_t + s_\theta(x_t, t)\right]\right) dt$$

4. Diffusion models can be considered CNFs trained with score matching!

Sampling from "continuous-Time" Diffusion models

How to solve the generative SDE or ODE in practice?

- Generative diffusion SDE:

$$dx_t = -\frac{1}{2}\beta(t)\left[x_t + 2s_\theta(x_t, t)\right]dt + \sqrt{\beta(t)}\, d\bar{w}_t$$

- Probability flow ODE:

$$dx_t = -\frac{1}{2}\beta(t)\left[x_t + s_\theta(x_t, t)\right]dt$$

→ Euler-Maruyama:

$$x_{t-1} = x_t + \frac{1}{2}\beta(t)\left[x_t + 2s_\theta(x_t, t)\right]\Delta t + \sqrt{\beta(t)\Delta t}\, N(0, I)$$

→ ancestral sampling is also a generative SDE sampler!

→ Euler's method:

$$X_{t-\tau} = X_t + \frac{1}{2}\beta(t)\left[X_t + s_\theta(X_t, t)\right]\Delta t$$

→ in practice: higher order ODE solvers
(Runge-Kutta, linear multistep methods,
exponential integrators, ...)

SDE:

$$dX_t = -\frac{1}{2}\beta(t)\left[X_t + 2s_\theta(X_t, t)\right]dt + \sqrt{\beta(t)}\,dw_t$$

↓ probability flow ODE

$$dX_t = -\frac{1}{2}\beta(t)\left[X_t + s_\theta(X_t, t)\right]dt - \frac{1}{2}\beta(t)\left[X_t + s_\theta(X_t, t)\right]dt + \sqrt{\beta(t)}\,dw_t$$

Langevin diffusion SDE

- pro: continuous noise injection can help to compensate errors during diffusion process.

- con, often slower, because the stochastic terms themselves require fine discretization during solve.

probability flow ODE

pro: can leverage fast ODE solvers. best when targeting very fast sampling.

con: no "stochastic" error correction, often slightly lower performance than stochastic sampling.

Diffusion models as Energy-based models

- Assume an Energy-based model (EBM):

$$p_\theta(x,t) = \frac{e^{-E_\theta(x,t)}}{Z_\theta(t)}$$

- Sample EBM via Langevin dynamics:

$$x_{i+1} = x_i - \eta \nabla_x E_\theta(x_i,t) + \sqrt{2\eta}\, \mathcal{N}(0,I)$$

- requires only gradient of energy $(-\nabla_x E_\theta(x,t))$, not $E_\theta(x,t)$ itself, nor $Z_\theta(t)$!

- in diffusion models, we learn "energy gradients" for all diffused distributions directly:

$$\nabla_x \log q_t(x) \approx s_\theta(x,t) =: \nabla_x \log p_\theta(x,t)$$

$$= -\nabla_x E_\theta(x,t) - \nabla_x \log Z_\theta(t)$$

$$= 0$$

$$= -\nabla_x E_\theta(x,t)$$

→ diffusion models model energy gradient directly, along entire diffusion process, and avoid modeling partition function. different noise levels along diffusion are ~~also~~ analogous to annealed sampling in EBMs.

unique Identifiability

forward diffusion SDE:

$$dx_t = -\frac{1}{2}\beta(t) x_t dt + \sqrt{\beta(t)}\, dw_t$$

reverse generative diffusion SDE:

$$dx_t = -\frac{1}{2}\beta(t)\underbrace{[x_t + 2\nabla_{x_t} \log q_t(x)]}_{\approx s_\theta(x_t,t)} dt + \sqrt{\beta(t)}\, d\bar{w}_t$$

- denoising model $s_\theta(x_t,t)$ and deterministic data encodings uniquely determined by data and fixed →

→ forward diffusion!

- Even with different architectures and initializations we recover identical model outputs and encodings (given sufficient training data, model capacity and optimization accuracy), in contrast to GANs, VAEs, etc.

Advanced Techniques: Accelerated Sampling, conditional Generation, and Beyond.

- what makes a good generative model?

1. fast sampling.
2. Mode coverage/Diversity.
3. High quality samples.

- GANs are good in ① and ③ but not in ②
- Likelihood-based models (VAEs & normalizing flows) are good at ① and ② but not in ③.
- diffusion models are good in ② and ③ but not ①

-

- How to accelerate diffusion models?

  - naiive acceleration methods:

    - reducing diffusion time steps in training
    - sampling every k time step in inference

    problems: lead to immediate worse performance.

  - given a limited number of functional calls, usually much less than 1000s, how to improve performance?

- forward diffusion process (markov chain)

$$x_0 \longrightarrow \cdots \longrightarrow x_t \longrightarrow x_{t+1} \longrightarrow \cdots \longrightarrow x_T$$

$$q(x_t | x_{t-1}) := \mathcal{N}(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I)$$

(?)

- Does the noise schedule have to be predefined?
- Does it have to be a Markovian process?
- is there any faster mixing diffusion process?

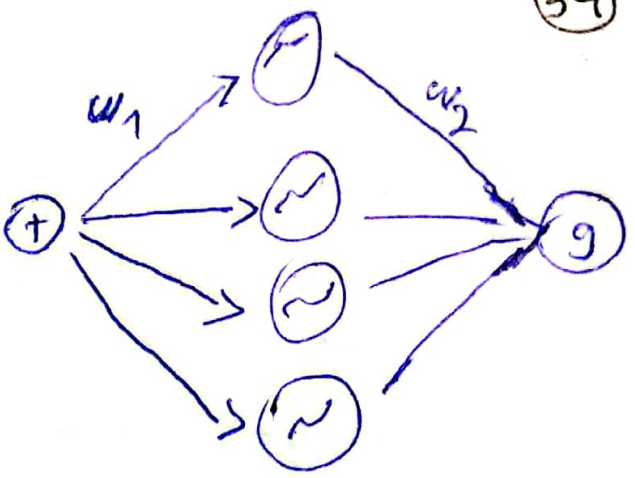
- variational diffusion models (learnable diffusion process)
  - given the forward process:

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}\, x_0; (1-\bar{\alpha}_t) I)$$

- directly parameterize the variance through a learned function γ_η: $1-\bar{\alpha}_t = \text{sigmoid}(\gamma_\eta(t))$

$\gamma_\eta(t)$: a monotonic MLP.

- strictly positive weights & monotonic activations (e.g., sigmoid)



- so, this means we optimize the parameters and use them in the decoder.

- the training objective of variational diffusion models:

$$L_T = \frac{T}{2} E_{x_0, \epsilon, t}\left[\left(\exp\left(\gamma_\eta(t) - \gamma_\eta(t-1)\right) - 1\right) \| \epsilon - \epsilon_\theta \hookrightarrow (x_t, t) \|_2^2\right]$$

- learning noise schedule improves likelihood estimation of diffusion models, given fewer diffusion steps.

- letting $T \to \infty$ leads to variational upper bound in continuous time:

$$L_\infty = \frac{1}{2} E_{x_0, \epsilon, t}\left[\gamma_\eta'(t) \| \epsilon - \epsilon_0(x_t, t) \|_2^2\right],$$

$$\gamma_\eta'(t) = \partial \gamma_\eta(t) / \partial t$$

Denoising diffusion implicit models (DDIM)

(non-markovian diffusion process)

main Idea

- design a family of non-markovian diffusion processes and corresponding reverse processes.
- The process is designed such that the model can be optimized by the same surrogate objective as the original diffusion model.

$$L_{simple}(\theta) := E_{t, x_0, \epsilon}\left[\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t) \|^2 \right]$$

- therefore, can take a pretrained diffusion model but with more choices of sampling procedure.

- How to define the non-markovian forward process?

Recall that the KL divergence in the variational upper bound can be written as:

$$L_{t-1} = D_{KL}(q(x_{t-1} | x_t, x_0) \| p_\theta(x_{t-1} | x_t))$$

$$= E_q\left[\frac{1}{2\sigma_t^2} \| \tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t) \|^2 \right] + c$$

$$\begin{cases} \tilde{\mu}_t(x_t, x_0) = \frac{1}{\sqrt{1-\beta_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon\right) \\ \mu_\theta(x_t, t) = \frac{1}{\sqrt{1-\beta_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right) \end{cases}$$

so;

$$L_{t-1} = E_{x_0 \sim q(x_0), \epsilon \sim N(0,I)}\left[\lambda_t \|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t)\|^2\right] + C \quad x_t$$

if we assume loss weighting λ_t can be arbitrary values, the above formulation holds as long as:

$$q(x_t|x_0) = N(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\alpha_t)I)$$

(make sure: $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{(1-\bar{\alpha}_t)}\epsilon$)

forward process: $q(x_{t-1}|x_t, x_0) = N(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\sigma}_t^2 I)$

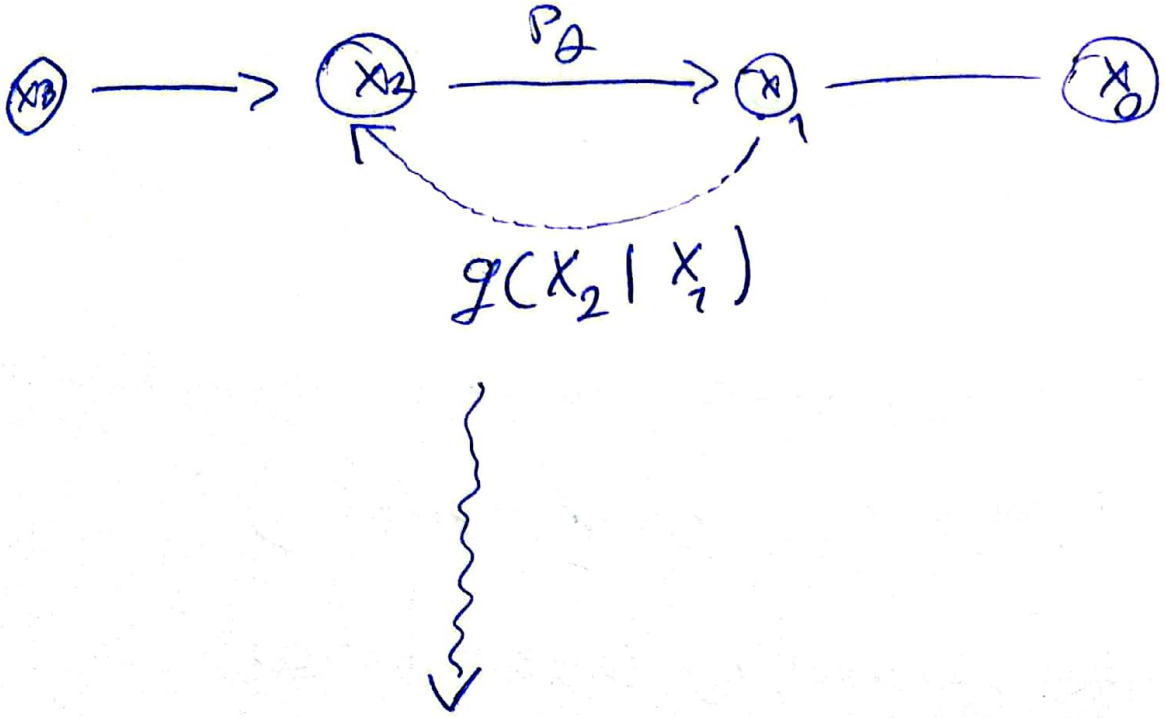
$$\tilde{\mu}_t(x_t, x_0) = ax_t + b\epsilon = ax_t + b\frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{\sqrt{1-\bar{\alpha}_t}}$$

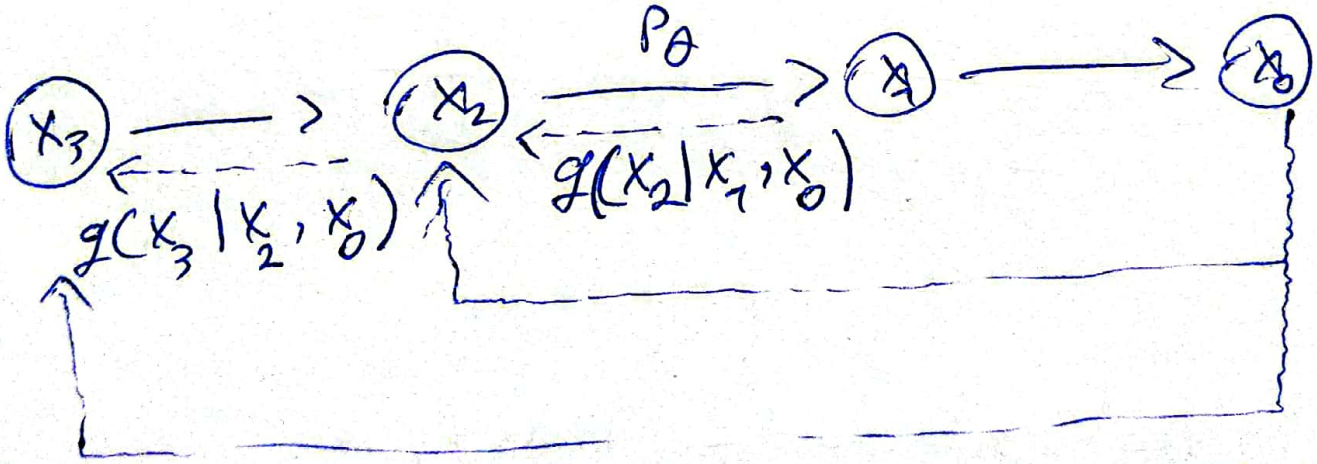
reverse process: $p_\theta(x_{t-1}|x_t) = N(x_{t-1}; \mu_\theta(x_t, t), \tilde{\sigma}_t^2 I)$

$$\mu_\theta(x_t, t) = ax_t + b\epsilon_\theta(x_t, t) = ax_t + b\frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{\sqrt{1-\bar{\alpha}_t}}$$

(assume: $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{(1-\bar{\alpha}_t)}\epsilon_\theta(x_t, t)$)

no need to specify $q(x_t | x_{t-1})$ to be a Markovian process!





- define a family of forward processes ~~that~~ q

$$q(x_{t-1} | x_t, x_0) = \mathcal{N}\left(\sqrt{\bar{a}_{t-1}}\, x_0 + \sqrt{1-\bar{a}_{t-1} - \tilde{\sigma}_t^2} \cdot \frac{x_t - \sqrt{\bar{a}_t}\, x_0}{\sqrt{1-\bar{a}_t}}, \tilde{\sigma}_t^2 I \right)$$

The corresponding reverse process is:

$$p(x_{t-1}|x_t) = \mathcal{N}\left(\sqrt{\bar{\alpha}_{t-1}}\,\hat{x}_0 + \sqrt{1-\bar{\alpha}_t-\tilde{\sigma}_t^2}\cdot \frac{x_t-\sqrt{\bar{\alpha}_t}\,\hat{x}_0}{\sqrt{1-\bar{\alpha}_t}}, \tilde{\sigma}_t^2 I\right)$$

compared to diffusion models much less diffusion steps (5 steps)

Diffusion energy-based models:

(conditional energy-based models)

- They can be derived by bayes' rule:

$$p_\theta(x|\tilde{x}) = \frac{1}{\tilde{Z}_\theta(\tilde{x})}\exp\left(f_\theta(x) - \frac{1}{2\sigma^2}\|\tilde{x}-x\|^2\right)$$

↓ localized the energy landscape.

- learn the sequence of EBMs by maximizing conditional log-likelihoods:

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n} \log p_\theta(x_i|\tilde{x}_i)$$

Advanced modeling

(latent space modeling & model ~~distribution~~ distillation)

(variational autoencoder + score-based prior)

(?) - can we do model distillation for fast sampling?

- can we lift the diffusion model to a latent space that is faster to diffuse?

"Applications of diffusion models"

(image synthesis, controllable generation, text-to-image)

- conditional generation:
  - given a text prompt c, generate high-res images x.

exps: GLIDE (OpenAI), DALL·E 2 (OpenAI)

Imagen (google research)